Dynamic Camera Credentials — Approach Document

Project: Eco-Series-Kitty

Date: March 2026

Status: Proposed

Components: MQTT Client (C), MQTT Server (Node.js), MongoDB (credentials collection)

1. Current Problem

> **Camera credentials are hardcoded in the C client and never sync with the database.**

1.1 Current State — Where Credentials Are Used

#	Location	Current Value	Problem

1	N_HTTPS.c:7-8
	`#define USERNAME "admin"`
	`#define PASSWORD ""`
		`admin` : `""` (empty)	Hardcoded. Used by all 50+ HTTP API calls to camera. Never updates.

2	HTTP.c:306 `send_PTZ_request()`	`YWRtaW46` = base64(`admin:`)	Hardcoded base64. PTZ commands always use `admin` with empty password.
3	HTTP.c:415 `send_https_request_case40()`	`YWRtaW46` = base64(`admin:`)	Hardcoded base64. Case 40 always uses `admin` with empty password.

4	HTTP.c:1391,1428,1854
	Cases 49, 50, 71
		`username=admin&password=`
		Case 50:
		`password=your_password`
			Hardcoded in URL params. Case 50 has a placeholder string.

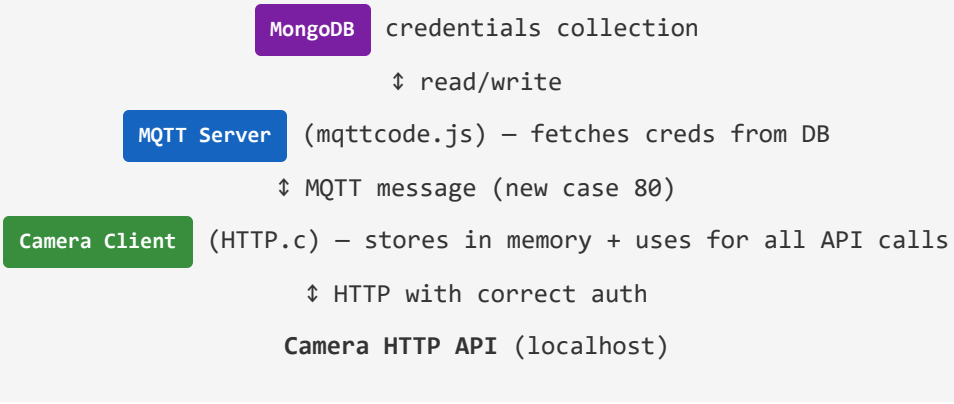
5	HTTP.c:92 `update_password()`	Updates runtime `PASSWORD` global	Only updates in-memory. Lost on reboot. N_HTTPS.c ignores it anyway.

1.2 Database — What's Available

```
// MongoDB: gamma-arcis → credentials collection
{
  "_id": { "$oid": "68e637a80232c703428be123" },
  "deviceId": "VSPL-148034-NJRWJ",
  "username": "admin",
  "password": "",
  "token": "a/b4Znt+OFGrYtmHw0T16Q==",
  "lastUpdatedAt": null
}
```

> **Key insight:** The VMS frontend/backend already stores camera credentials per device in MongoDB. The MQTT server can access this database. The camera (C client) cannot access MongoDB directly — it must receive credentials via MQTT.

2. Proposed Solution — Architecture



2.1 The Approach — 3 Parts

Part	Component	What Changes
Part A Server → Camera (credential delivery)	**MQTT Server**	1. On camera connect → fetch credentials from MongoDB `credentials` collection by `deviceId` 2. Send credentials to camera via new MQTT topic `torque/rx/{deviceId}/80` 3. On case 71 (password change) → update MongoDB after camera confirms success
Part B Camera stores & uses (credential consumption)	**MQTT Client**	1. New global `CAM_USERNAME[64]` and `CAM_PASSWORD[64]` for camera HTTP auth 2. Case 80 handler — receives credentials from server, stores in globals 3. N_HTTPS.c `send_https_request()` uses these globals instead of hardcoded values 4. `send_PTZ_request()` and `send_https_request_case40()` build dynamic base64 auth 5. Cases 49, 50, 71 use dynamic credentials in URL params
Part C Password change sync (credential update)	**Server** + **Client**	1. App sends password change (case 71) → camera changes password 2. Camera confirms success → publishes new password on `torque/tx/{deviceId}/71` 3. Server receives confirmation → updates MongoDB `credentials` collection 4. Camera updates runtime `CAM_PASSWORD` + rebuilds base64 auth

3. Detailed Design

3.1 Part A — Server: Credential Delivery

> **New:** When camera connects, server fetches credentials from MongoDB and sends them via MQTT.

New MongoDB Model: `models/credential.js`

```
const mongoose = require('mongoose');

const credentialSchema = new mongoose.Schema({
  deviceId: { type: String, required: true, unique: true },
  username: { type: String, default: 'admin' },
  password: { type: String, default: '' },
  token: { type: String },
  lastUpdatedAt: { type: Date, default: null }
});

module.exports = mongoose.model('Credential', credentialSchema);
```

Server: On Camera Connect (mqttcode.js)

```
// In aedes.on('client', ...) handler — after the existing NTP push (case 23)

if (!client.id.startsWith("mqttjs")) {
  // Existing: push NTP config (case 23)
  ...

  // NEW: Send camera credentials (case 80)
  try {
    const cred = await Credential.findOne({ deviceId: client.id });
    if (cred) {
      const credPayload = JSON.stringify({
        username: cred.username,
        password: cred.password
      });
      publishMessage(`torque/rx/${client.id}/80`, credPayload);
      console.log(`Sent credentials to ${client.id}`);
    } else {
      // No credentials in DB — send defaults
      publishMessage(`torque/rx/${client.id}/80`, JSON.stringify({
        username: "admin",
        password: ""
      }));
    }
  } catch (err) {
    console.error(`Failed to fetch credentials for ${client.id}:`, err);
  }
}
```

Server: On Password Change Success (case 71 response)

```
// In case '71' handler in the publish event

case '71':
  const deviceIdCred = topicSuffix.split('/')[0];
  // Forward to app
  publishMessage(sendTOAppTopic, packet.payload);

  // NEW: Update credentials in DB if password change was successful
  try {
    const response = JSON.parse(packet.payload.toString());
    if (response && response.new_password) {
      await Credential.updateOne(
        { deviceId: deviceIdCred },
        { $set: { password: response.new_password, lastUpdatedAt: new Date() } },
        { upsert: true }
      );
      console.log(`Updated credentials in DB for ${deviceIdCred}`);
    }
  } catch (err) {
    console.error(`Failed to update credentials for ${deviceIdCred}:`, err);
  }
  break;
```

3.2 Part B — Client: Credential Storage & Usage

> **New:** Camera receives credentials via case 80 and uses them for all HTTP API calls.

HTTP.c — New Global Variables

```
// Camera HTTP API credentials (received from server via case 80)
char CAM_USERNAME[64] = "admin";    // default
char CAM_PASSWORD[64] = "";         // default empty
char CAM_AUTH_BASIC[256] = "Basic YWRtaW46"; // base64(admin:) — rebuilt on credential update
```

HTTP.c — New Function: Rebuild Base64 Auth

```
// Build "Basic base64(username:password)" header from current credentials
void rebuild_basic_auth(void) {
    char raw[128];
    snprintf(raw, sizeof(raw), "%s:%s", CAM_USERNAME, CAM_PASSWORD);

    // Base64 encode
    static const char b64[] =
"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/";
    char encoded[256];
    int i = 0, j = 0, len = strlen(raw);
    // ... standard base64 encoding ...
```

```
    snprintf(CAM_AUTH_BASIC, sizeof(CAM_AUTH_BASIC), "Basic %s", encoded);
}
```

HTTP.c — Case 80: Receive Credentials

```
case 80: {
    printf("Handling case 80: credential update\n");
    cJSON *root = cJSON_Parse(safePayload);
    if (!root) { printf("JSON parse error\n"); break; }

    cJSON *user = cJSON_GetObjectItem(root, "username");
    cJSON *pass = cJSON_GetObjectItem(root, "password");

    if (cJSON_IsString(user))
        snprintf(CAM_USERNAME, sizeof(CAM_USERNAME), "%s", user->valuestring);
    if (cJSON_IsString(pass))
        snprintf(CAM_PASSWORD, sizeof(CAM_PASSWORD), "%s", pass->valuestring);

    rebuild_basic_auth();
    printf("Camera credentials updated: user=%s\n", CAM_USERNAME);

    cJSON_Delete(root);
    break;
}
```

N_HTTPS.c — Use Dynamic Credentials

```
// BEFORE (hardcoded):
#define USERNAME "admin"
#define PASSWORD ""
curl_easy_setopt(curl, CURLOPT_USERNAME, USERNAME);
curl_easy_setopt(curl, CURLOPT_PASSWORD, PASSWORD);

// AFTER (dynamic — passed from HTTP.c):
extern char CAM_USERNAME[64];
extern char CAM_PASSWORD[64];
curl_easy_setopt(curl, CURLOPT_USERNAME, CAM_USERNAME);
curl_easy_setopt(curl, CURLOPT_PASSWORD, CAM_PASSWORD);
```

send_PTZ_request & send_https_request_case40 — Dynamic Base64

```
// BEFORE (hardcoded):
"Authorization: Basic YWRtaW46\r\n"

// AFTER (dynamic):
extern char CAM_AUTH_BASIC[256];

// In the HTTP request string:
snprintf(http_request, sizeof(http_request),
    "PUT ... HTTP/1.1\r\n"
```

```
    "Authorization: %s\r\n"   // dynamic
    ..., CAM_AUTH_BASIC);
```

3.3 Part C — Password Change Sync

Password Change Flow (Case 71)

1. App sends {"old_pass":"", "new_pass":"secret123"}

↓ MQTT torque/rx/{SN}/71

2. **Server** relays to camera (existing behavior)

↓

3. **Camera** changes password on camera HTTP API

↓ success

4. **Camera** updates CAM_PASSWORD + rebuilds base64 auth
5. **Camera** publishes {"status":"success", "new_password":"secret123"}

↓ MQTT torque/tx/{SN}/71

6. **Server** receives → updates **MongoDB** credentials collection
7. **Server** forwards to App (existing behavior)

On next reboot:

Camera connects → Server sends stored credentials (case 80) → Camera uses them

4. Sequence Diagram — Full Lifecycle

```
BOOT / CONNECT:
Camera ──connect──→ MQTT Server
                    Server: fetch credentials from MongoDB by deviceId
                    Server ──case 80──→ Camera: {"username":"admin","password":"secret123"}
                    Camera: stores CAM_USERNAME, CAM_PASSWORD, rebuilds base64
                    Camera: all HTTP API calls now use correct credentials

NORMAL OPERATION:
Camera ──HTTP API──→ Camera localhost (using CAM_USERNAME:CAM_PASSWORD)
                     All 50+ cases work with correct auth

PASSWORD CHANGE (Case 71):
App ──case 71──→ Server ──relay──→ Camera
                                   Camera: changes password on camera HTTP API
                                   Camera: updates CAM_PASSWORD + rebuild base64
Camera ──case 71 response──→ Server: updates MongoDB credentials
                             Server ──relay──→ App
```

```
REBOOT:
Camera reconnects → Server sends stored credentials (case 80) → works with new password
```

5. Files to Modify

File	Changes	Effort
mqtt-client/HTTP.c	• Add `CAM_USERNAME`, `CAM_PASSWORD`, `CAM_AUTH_BASIC` globals • Add `rebuild_basic_auth()` function • Add case 80 handler • Update case 71 to publish new password on success • Update `send_PTZ_request()` to use `CAM_AUTH_BASIC` • Update `send_https_request_case40()` to use `CAM_AUTH_BASIC` • Update cases 49, 50, 71 URL params to use `CAM_USERNAME/CAM_PASSWORD`	Medium
mqtt-client/N_HTTPS.c	• Remove `#define USERNAME/PASSWORD` • Use `extern CAM_USERNAME/CAM_PASSWORD` from HTTP.c	Small
mqtt-server/mqttcode.js	• Connect to VMS MongoDB (`gamma-arcis`) • On camera connect: fetch + send credentials (case 80) • On case 71 response: update credentials in DB	Small
mqtt-server/models/credential.js **NEW**	Mongoose model for `credentials` collection	Small
mqtt-server/config/db.js	Add second MongoDB connection to VMS database	Small
mqtt-server/utils/commonConfig.js	Add `MSG_TYPE_CREDENTIAL_SYNC: '80'`	Trivial

6. Security Considerations

Concern	Mitigation
Credentials sent in plaintext over MQTT	MQTT broker supports TLS on port 8883. Camera should connect via TLS. Currently using plain TCP 1883 — consider migrating to 8883.
Credentials visible in camera logs	Never log the password value. Log `"credentials updated for user=admin"` not the password itself.
MongoDB connection string in code	Store VMS DB URI in `.env` file (already gitignored). Use `process.env.DBURIVMS`.
Token field in credentials collection	The `token` field appears to be encrypted. Future: use token-based auth instead of plaintext password.
Camera reboots — credentials lost	Server resends credentials on every connect (case 80). No persistence needed on camera.

7. Implementation Order

Step	What	Description
Step 1	Client: Add credential globals + case 80	Add `CAM_USERNAME`, `CAM_PASSWORD`, `CAM_AUTH_BASIC`, `rebuild_basic_auth()`, and case 80 handler in HTTP.c
Step 2	Client: Update all API calls	N_HTTPS.c uses extern globals. Update `send_PTZ_request`, `send_https_request_case40`, cases 49/50/71 URL params
Step 3	Server: Add credential model + DB connection	Add `credential.js` model, connect to VMS MongoDB, add `MSG_TYPE_CREDENTIAL_SYNC`
Step 4	Server: Send credentials on connect	In `aedes.on('client')` handler, fetch from DB and publish case 80
Step 5	Server: Sync on password change	In case 71 response handler, update MongoDB credentials collection
Step 6	Test end-to-end	Deploy new client + server, verify credentials flow on test camera

8. Testing Plan

Test	Steps	Expected Result
Camera boot with DB credentials	1. Set password in MongoDB for test device 2. Reboot camera	Camera receives credentials via case 80, API calls use correct password
Camera boot with no DB entry	1. Remove device from credentials collection 2. Reboot camera	Camera receives default `admin:""`, works with factory camera
Password change via app	1. Send case 71 with new password 2. Check MongoDB updated 3. Send case 7 (MAC address) to verify API still works	Password changed, DB updated, subsequent API calls succeed
Reboot after password change	1. Change password via case 71 2. Reboot camera 3. Check API calls work	Server sends new password from DB on reconnect, camera uses it
PTZ with non-default password	1. Set password in DB 2. Send PTZ command (case 51)	PTZ works — base64 auth header uses correct credentials

---

Document: Dynamic Camera Credentials Approach | **Author:** Rahul Vadhiya | **Date:** March 2026